

Business Rule Audit Report

Source: **main.clear**

Generated: 2026-05-05T02:33:57.162Z

This report shows every named business rule in the source application, how each was proved formally by the math-checker, and how each was verified at runtime by sending violating inputs to the running application and observing rejections.

3/3	0/0	0	0
RULES PROVED by structural enforcement	VIOLATING INPUTS rejected at runtime	RULES DISPROVED (should be 0)	UNVERIFIABLE (impure body)

Trust basis: how to read this report

Math-proved. The compiler's prover walks every named rule and decides whether the rule holds for every possible input. Two paths are possible: (a) the guard expression simplifies to a constant true (a tautology), or (b) the guard exists as a control-flow check — any execution past the guard satisfies the condition because the runtime rejects any input that fails it before control reaches the next line.

Runtime-verified. The math claim depends on the compiler correctly emitting the runtime guard. To corroborate, this report spawns the compiled application on a free port and sends 20 inputs that violate the rule. If every input is rejected with HTTP 403 and every response body carries the rule's name, the math claim is corroborated. If even one input slips through with a 200 response, the math claim is not trustworthy.

Two-witness verification. A rule earns full audit confidence only when BOTH witnesses agree — math says it's impossible to violate, AND runtime measurably rejects violators. The metrics row at the top shows both counts.

Rule-by-rule audit

Rule: discount-cap-thirty

VERDICT: PROVED

How it was proved formally

This rule is enforced by construction of the program, not by math simulation. The math-checker can't simulate every possible deal — the values are not bounded. So instead it reads the structure of the compiled application and confirms that the compiler put a hard check at line 127 that rejects any deal that fails the condition.

Your source (line 127):

```
enforce that deal's discount_percent is less than 30, or fail with error message: 'Discounts of 30% or more require VP approval – submit through the VP escalation queue, not the standard CRO desk'
```

The actual runtime check (compiled from line 127):

```
if (!(deal?.discount_percent < 30)) { return res.status(403).json({ error: "Discounts of 30% or more require VP approval – submit through the VP escalation queue, not the standard CRO desk", rule: "discount-cap-thirty" }); }
```

The claim, in plain English: no line of compiled code after this check ever runs for a deal that fails the condition. Not because the math-checker proved it abstractly, but because the program literally rejects those inputs before reaching the next line. The runtime evidence below corroborates that the rejection actually fires.

How it was verified at runtime

Runtime witness automation pending for this rule shape.

runtime witness skipped: the compiled application needs `jsonwebtoken` installed to boot. The math proof still stands; install dependencies and re-run the audit for runtime corroboration.

Rule: price-floor-positive

VERDICT: PROVED

How it was proved formally

This rule is enforced by construction of the program, not by math simulation. The math-checker can't simulate every possible deal — the values are not bounded. So instead it reads the structure of the compiled application and confirms that the compiler put a hard check at line 130 that rejects any deal that fails the condition.

Your source (line 130):

```
enforce that deal's list_price is greater than 0, or fail with error message: 'List price must be positive – zero or negative prices are never valid'
```

The actual runtime check (compiled from line 130):

```
if (!(deal?.list_price > 0)) { return res.status(403).json({ error: "List price must be positive – zero or negative prices are never valid", rule: "price-floor-positive" }); }
```

The claim, in plain English: no line of compiled code after this check ever runs for a deal that fails the condition. Not because the math-checker proved it abstractly, but because the program literally rejects those inputs before reaching the next line. The runtime evidence below corroborates that the rejection actually fires.

How it was verified at runtime

Runtime witness automation pending for this rule shape.

runtime witness skipped: the compiled application needs `jsonwebtoken` installed to boot. The math proof still stands; install dependencies and re-run the audit for runtime corroboration.

Rule: discount-not-over-cap

VERDICT: PROVED

How it was proved formally

This rule is enforced by construction of the program, not by math simulation. The math-checker can't simulate every possible deal — the values are not bounded. So instead it reads the structure of the compiled application and confirms that the compiler put a hard check at line 133 that rejects any deal that fails the condition.

Your source (line 133):

```
enforce that deal's discount_percent is less than 100, or fail with error message: 'Discount cannot meet or exceed 100% – that would mean giving the product away or paying the customer'
```

The actual runtime check (compiled from line 133):

```
if (!(deal?.discount_percent < 100)) { return res.status(403).json({ error: "Discount cannot meet or exceed 100% – that would mean giving the product away or paying the customer", rule: "discount-not-over-cap" }); }
```

The claim, in plain English: no line of compiled code after this check ever runs for a deal that fails the condition. Not because the math-checker proved it abstractly, but because the program literally rejects those inputs before reaching the next line. The runtime evidence below corroborates that the rejection actually fires.

How it was verified at runtime

Runtime witness automation pending for this rule shape.

runtime witness skipped: the compiled application needs `jsonwebtoken` installed to boot. The math proof still stands; install dependencies and re-run the audit for runtime corroboration.